



Full Name: Jianping Ye

Email: yehdan@msn.com

Test Name: **Mock Test**

Taken On: 6 Aug 2025 21:02:04 IST

Time Taken: 25 min 29 sec/ 33 min

Invited by: Ankush

Invited on: 6 Aug 2025 21:01:56 IST

Skills Score:

Tags Score:

Algorithms 120/120

Core CS 120/120

Data Structures 120/120

Queues 120/120

100%

120/120

scored in **Mock Test** in 25 min
29 sec on 6 Aug 2025 21:02:04
IST

Recruiter/Team Comments:

No Comments.

	Question Description	Time Taken	Score	Status
Q1	Truck Tour > Coding	24 min 11 sec	120/ 120	✓

QUESTION 1

✓

Correct Answer

Score 120

Truck Tour > Coding

AlgorithmsData StructuresQueuesCore CS

QUESTION DESCRIPTION

Suppose there is a circle. There are N petrol pumps on that circle. Petrol pumps are numbered 0 to $(N - 1)$ (both inclusive). You have two pieces of information corresponding to each of the petrol pump: (1) the amount of petrol that particular petrol pump will give, and (2) the distance from that petrol pump to the next petrol pump.

Initially, you have a tank of infinite capacity carrying no petrol. You can start the tour at any of the petrol pumps. Calculate the first point from where the truck will be able to complete the circle. Consider that the truck will stop at each of the petrol pumps. The truck will move one kilometer for each litre of the petrol.

Input Format

The first line will contain the value of N .

The next N lines will contain a pair of integers each, i.e. the amount of petrol that petrol pump will give and the distance between that petrol pump and the next petrol pump.

Constraints:

$1 \leq N \leq 10^5$

$1 \leq \text{amount of petrol, distance} \leq 10^9$

Output Format

An integer which will be the smallest index of the petrol pump from which we can start the tour.

Sample Input

```
3
1 5
10 3
3 4
```

Sample Output

```
1
```

Explanation

We can start the tour from the second petrol pump.

CANDIDATE ANSWER

Language used: **C++14**

```
1
2
3 void pet_sum(vector<vector<int>> pumps, vector<int> &psum)
4 {
5     // Calculate poup 0 => i
6     int n = pumps.size();
7     int sum = 0;
8     for(int i=0;i<n;i++) {
9         sum += pumps[i][0] - pumps[i][1];
10        // from 0 to i+1
11        psum.push_back(sum);
12    }
13 }
14
15 // Both i, j in [0, n-1]
16 int pet_sum_from_to(vector<int> psum, int i, int j)
17 {
18     if (i==j) {
19         // round trip
20         int n = psum.size();
21         // [i, n-1] + [n-1, 0] + [0, i]
22         return psum[n-1];
23     }
24     else if (i==0) {
25         // [0, j]
26         return psum[j-1];
27     }
28     else if (i<j) {
29         // [i, j]
30         return psum[j-1] - psum[i-1];
31     }
32     else {
33         int n = psum.size();
34         // i>j => [i, n-1] + [n-1, 0] + [0, j]
35         return psum[n-1] - psum[i-1] + psum[j-1];
36     }
37 }
```

```

38 int tour_slow(vector<vector<int>> pumps)
39 {
40     vector<int> psum;
41     pet_sum(pumps, psum);
42
43     int n = psum.size();
44     for (int i=0;i<n;i++) {
45         bool pass = true;
46         for (int j=i+1; j<=i+n;j++) {
47             int jd = j%n;
48             if (pet_sum_from_to(psum, i, jd)<0) {
49                 pass = false;
50                 break;
51             }
52         }
53         if (pass) return i;
54     }
55     return -1;
56 }
57
58 int tour_fast(vector<vector<int>> pumps)
59 {
60     int n = pumps.size();
61
62     int start_index = 0;
63     int current_petrol = 0;
64
65     for (int i=0;i<n;i++) {
66         current_petrol += pumps[i][0] - pumps[i][1];
67
68         if (current_petrol<0) {
69             start_index = i+1;
70             current_petrol = 0;
71         }
72     }
73     return start_index;
74 }
75
76
77 /*
78  * Complete the 'truckTour' function below.
79  *
80  * The function is expected to return an INTEGER.
81  * The function accepts 2D_INTEGER_ARRAY petrolpumps as parameter.
82  */
83
84 int truckTour(vector<vector<int>> petrolpumps) {
85     return tour_fast(petrolpumps);
86 }
87
88

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	 Success	0	0.0095 sec	8.5 KB
Testcase 2	Easy	Hidden case	 Success	10	0.0101 sec	8.75 KB
Testcase 3	Easy	Hidden case	 Success	10	0.013 sec	8.63 KB
Testcase 4	Easy	Hidden case	 Success	10	0.0114 sec	8.75 KB
Testcase 5	Easy	Hidden case	 Success	10	0.0592 sec	24 KB

Testcase 6	Easy	Hidden case		Success	10	0.097 sec	24 KB
Testcase 7	Easy	Hidden case		Success	10	0.0683 sec	24 KB
Testcase 8	Easy	Hidden case		Success	10	0.0583 sec	23.9 KB
Testcase 9	Easy	Hidden case		Success	10	0.0962 sec	23.9 KB
Testcase 10	Easy	Hidden case		Success	10	0.0785 sec	24.1 KB
Testcase 11	Easy	Hidden case		Success	10	0.0589 sec	24.1 KB
Testcase 12	Easy	Hidden case		Success	10	0.0711 sec	24.3 KB
Testcase 13	Easy	Hidden case		Success	10	0.0679 sec	24 KB

No Comments